

DeliveryApp - 5 to 10 Minute Presentation Script

Slide 1 - Project Introduction

DeliveryApp is a food delivery platform inspired by Uber Eats.

It is built around three actors: Customer, Courier, and Admin.

The objective was practical: deliver a working end-to-end product first, then optimize architecture and quality.

Slide 2 - Project Objectives

The project has five main objectives:

1. Build a working delivery workflow from order creation to final review.
 2. Enforce role-based access and business rules with secure APIs.
 3. Separate concerns with clear backend layers.
 4. Keep the system scalable through a modular monolith migration path.
 5. Provide a usable frontend for real interactions, not only API simulation.
-

Slide 3 - What the App Does Today

Current functional scope:

- Customer: browse stores/products, place orders, track status, pay, review.
- Courier: view available orders, self-assign delivery, update order status.
- Admin: manage stores/products/addresses, moderate user accounts.

So this is already a functional platform, not only a technical proof of concept.

Slide 4 - Architecture Overview

High-level architecture:

- Blazor frontend for role-based user experience.
- ASP.NET Core Minimal API as backend entry point.
- Application and Service layers for use cases and validation.
- Infrastructure layer with EF Core repositories.
- SQL Server relational database.

This layered design keeps business logic clean and API behavior predictable.

Slide 5 - Architecture Details and Design Choices

Important architectural decisions:

- JWT authentication with role policies (`CustomerOnly`, `CourierOnly`, `AdminOnly`).
- Specialized services and repositories replaced the old fat service/repository pattern.
- Legacy runtime is stable, while module adapters under `/api/modules/*` support gradual migration.

Result: the system works now and remains evolvable.

Slide 6 - Data Model Overview

Core entities:

- `Customers`, `Addresses`
- `Stores`, `Products`
- `Orders`, `OrderItems`
- `Couriers`
- `Payments`, `Reviews`
- `UserAccounts` (authentication and role identity)

The central entity is `Orders`, linking customer, store, delivery address, and optional courier.

Slide 7 - Data Model Integrity Rules

Key data and business constraints:

- `UserAccounts.Email` is unique.
- Order total comes from order items.
- Status transitions are controlled step by step.
- Payment amount must match order total.
- One review per order, only after delivery.
- Resource access is restricted by role and ownership.

This keeps data consistent and business-safe.

Slide 8 - End-to-End Demo Flow

A complete demo flow:

1. Login as Customer.
2. Create an order from store products.
3. Switch to Courier, assign and progress delivery.
4. Switch back to Customer and show courier details (name, phone, vehicle).
5. Complete payment and submit review.
6. Login as Admin and show store/account management.

This demonstrates cross-role coordination in one lifecycle.

Slide 9 - Current Status

Current state of the project:

- Backend operational.
- Frontend operational.
- Database model and scripts available.
- Main delivery lifecycle implemented across all roles.

At this stage, the core objective is achieved: a working application.

Slide 10 - Next Steps and Closing

Next priorities:

1. Add automated integration and end-to-end tests.
2. Complete migration from legacy endpoints to module endpoints.
3. Strengthen production readiness (secrets, monitoring, CI/CD).
4. Improve UX and real-time delivery experience.

Closing line: DeliveryApp is already functional and structured, with a clear path toward production quality.